MODULE 05 · AI PLATFORM ENGINEERING

WEEK 3 · LESSON 3

BITS Pilani DIGITAL

Excellence made yours

Advanced Pipeline Orchestration

Multi-environment workflows · Kubeflow · Ray · MLflow
Automated retraining · Drift detection · Model versioning

- Topic 1: Multi-Stage & Multi-Environment Orchestration
- Topic 2: Kubeflow Pipelines
- Topic 3: Ray & MLflow for Orchestration
- Topic 4: Automated Retraining & Drift Detection
- Topic 5: Hands-on Setup | Topic 6: Summary

Lesson Overview

Lesson 3 · Advanced Pipeline Orchestration

6	16	~70	INT
Topics	**Concept Blocks**	**Duration**	**Level**
Core areas	Short explainers	Total minutes	Intermediate

LEARNING OBJECTIVES

1. Design multi-stage, multi-environment ML orchestration workflows
2. Compare and use Kubeflow, Ray, and MLflow for pipeline orchestration
3. Implement automated retraining, drift detection, and model versioning

TOPIC 0

00

Lesson Introduction

What to Expect This Week

What to Expect This Week

Building on Week 2 Airflow foundations — going production-grade

🔀 Multi-Environment Pipelines

Dev → staging → prod with config management, isolation, and parameterised DAG generation

🐳 Kubeflow Pipelines

Kubernetes-native ML orchestration — architecture, SDK, and Airflow comparison

⚡ Ray & MLflow

Distributed compute with Ray, experiment tracking and registry with MLflow

🔄 Automated Retraining

Drift detection triggers, concept vs data drift, versioning and automated rollback

TOPIC 1

01

Multi-Stage & Multi-Environment Orchestration

Dev, staging, and production — each with its own config, gates, and IAM boundary

Multi-Stage Pipelines — Dev, Staging, and Production

Three environments, each with its own data, compute, and approval gate

BITS Pilani DIGITAL
Excellence made yours

DEV

Experiment & iterate

- Frequent retraining, no SLA
- MLflow / SageMaker Experiments tracking
- Data scientists own this tier
- S3 prefix: project/dev/

STAGING

Validate & gate

- Mirror of prod infrastructure
- Accuracy + latency gate checks
- Shadow-mode traffic testing
- S3 prefix: project/staging/

PROD

Serve & monitor

- Champion model only
- Full CloudWatch monitoring
- Drift detection runs live
- S3 prefix: project/prod/

Promotion rule: accuracy ≥ threshold · latency ≤ SLA · no critical drift → s3.copy_object() to next tier

Managing Dependencies and Configs Across Environments

One codebase — environment behaviour controlled by config, not code

BITS Pilani DIGITAL
Excellence made yours

⚙️ Config Injection

- Env vars: export ENV=staging
- Airflow Variables per deployment
- AWS SSM: /project/dev/threshold
- S3 config files per tier

Golden rule: same code, different config

📌 Dependency Pinning

- requirements.txt with exact versions
- sklearn==1.4.2, not sklearn>=1.4
- Docker images: tagged not :latest
- Separate dev / prod requirements

⚠️ :latest in prod = silent breakage

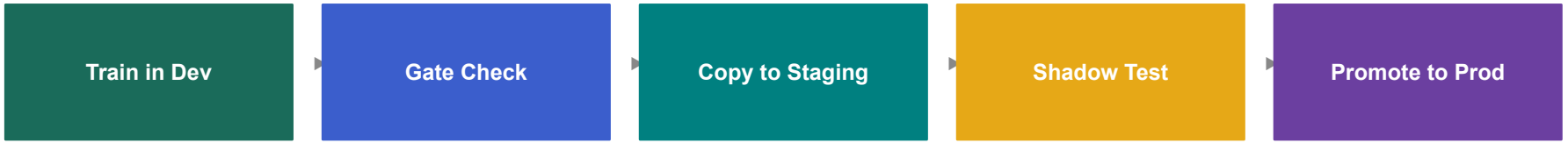
🔐 Secrets & IAM

- Never hardcode credentials in code
- Use AWS Secrets Manager at runtime
- Separate IAM role per environment
- Dev cannot access prod S3 buckets

Least privilege = smallest blast radius

Environment Isolation and Promotion Strategies

Model artifacts travel safely from dev to production via S3 copy + gate



🔒 Isolation Rules

- Separate IAM role per tier (dev cannot touch prod)
- Dedicated S3 prefix per environment
- CloudWatch namespace per tier
- No shared databases between dev and prod

🚪 Promotion Pattern

- Artifact promotion: s3.copy_object() — not code deploy
- Accuracy and latency gate enforced automatically
- Human approval required only for prod in regulated industries
- Rollback = re-promote previous approved version

🏷️ Lineage Tagging

- Every artifact tagged with: run_id, accuracy, drift_count
- Model Registry records which version is in which tier
- Experiment run links artifact to training data version
- Audit trail required by ML governance frameworks

Pipeline Parameterisation and Dynamic DAG Generation

One DAG definition — different configs injected at runtime

```
# Airflow: trigger with config at runtime
airflow dags trigger loan_risk_pipeline \
  --conf '{"env":"staging","threshold":0.82}'

# Inside the DAG — read the config
env       = conf.get('env', 'dev')
threshold = float(conf.get('threshold', 0.80))

# Dynamic DAG generation — one template, many DAGs
for model_type in ['rf', 'gbt', 'xgb']:
    dag_id = f'train_{model_type}'
    globals()[dag_id] = create_training_dag(
        model_type=model_type,
        env=env
    )
```

✅ Same codebase runs in all environments

Config injected at trigger time — no code changes needed for staging or prod runs.

✅ Backfills and A/B tests made easy

Re-trigger with a past date range or a different model config — same DAG handles it.

✅ Dynamic DAGs scale to many pipelines

Generate one DAG per model type, region, or business unit from a single template.

⚠️ Caution with dynamic DAGs

Dynamic generation makes UI harder to read. Use only when pipeline count > 5 and varies.

TOPIC 2

02

Kubeflow Pipelines

Kubernetes-native ML workflow orchestration at scale

Kubeflow Architecture and Core Components

ML-specific orchestration built natively on Kubernetes

🔗 Kubeflow Pipelines

Core orchestration layer. Defines ML workflows as containerised DAGs. Each step runs in its own K8s Pod with resource isolation.

🔍 Katib

Hyperparameter tuning and Neural Architecture Search. Automates grid search, Bayesian optimisation, and early stopping (ASHA).

🚀 KServe

Model serving layer. Deploys trained models as scalable REST/gRPC endpoints on Kubernetes with autoscaling and canary rollout.

📓 Notebook Server

Managed JupyterLab running inside K8s. Direct access to pipeline artifacts, storage, and cluster compute from the notebook.

🗄️ ML Metadata (MLMD)

Tracks all pipeline executions, step inputs/outputs, and artifact lineage automatically across every run.

🖥️ Central Dashboard

Unified web UI for all pipeline runs, experiment results, hyperparameter sweeps, and model serving endpoints.

Building ML Pipelines with Kubeflow SDK

Decorate Python functions — Kubeflow handles containerisation

```
from kfp import dsl
from kfp.dsl import component, pipeline

@component(packages_to_install=['sklearn'])
def train(data_path: str) -> float:
    # runs inside a container automatically
    ...
    return accuracy

@component
def evaluate(accuracy: float) -> str:
    return 'pass' if accuracy > 0.80 else 'fail'

@pipeline(name='loan-risk-pipeline')
def loan_pipeline(data_path: str):
    train_op = train(data_path)
    eval_op  = evaluate(train_op.output)

# Compile and submit
compiler.Compiler().compile(loan_pipeline,
'pipeline.yaml')
client.create_run_from_pipeline_func(loan_pipeline, {})
```

`@component`

A single Python function packaged into a Docker container automatically. Add packages_to_install for deps.

`@pipeline`

Wires components together using typed outputs. Kubeflow records the full execution graph in MLMD.

`Typed Artifacts`

Use dsl.Input[dsl.Dataset] and dsl.Output[dsl.Model] for typed I/O — lineage tracked automatically.

`ContainerOp`

Advanced: bring your own Docker image for full control. Useful when the function pattern is too limiting.

Kubeflow vs Airflow — Strengths, Tradeoffs, and Migration Paths

Choosing the right orchestrator for your ML workload

BITS Pilani DIGITAL
Excellence made yours

Dimension	Apache Airflow	Kubeflow Pipelines
Execution target	Any machine with Airflow workers	Kubernetes pods (always)
ML artifact tracking	Custom (XComs or external MLflow)	Built-in MLMD — automatic lineage
Hyperparameter tuning	External (Optuna, Ray Tune via operator)	Katib — native HPO integration
GPU scheduling	Via K8sExecutor resource limits	Native K8s GPU node affinity
Model serving	Deploy to external endpoint separately	KServe on the same cluster
Learning curve	Moderate — Python DAGs, operators	High — K8s, containers, KFP SDK
Team fit	Data engineering + ML teams together	Dedicated ML platform team
Best for	ETL + ML, general orchestration, small teams	Large ML teams on K8s at scale

BITS Pilani DIGITAL
Excellence made yours

TOPIC 3

03

Ray & MLflow for Orchestration

Distributed computing and experiment tracking for production ML

Distributed Computing with Ray — When and Why

BITS Pilani DIGITAL
Excellence made yours

Scale Python to a cluster without rewriting your code

```python
import ray
ray.init()  # or connect to cluster

# Decorate any function → runs on any worker
@ray.remote
def train_fold(fold_id, data):
    # executes in parallel across cluster
    return accuracy

# Launch 5 parallel training jobs
futures = [
    train_fold.remote(i, folds[i])
    for i in range(5)
]
results = ray.get(futures)  # wait for all

# Ray Tune: 200-trial HPO in parallel
tuner = tune.Tuner(train_fn, param_space={
    'lr': tune.loguniform(1e-4, 1e-1)
})
```

⚡ Ray Core

Parallel task execution with @ray.remote. Any Python function runs across a cluster without code changes.

📊 Ray Tune

Hyperparameter optimisation at scale. 200+ trials in parallel. Early stopping with ASHA or PBT schedulers.

🏋 Ray Train

Distributed model training across GPUs. Native integration with PyTorch, TensorFlow, XGBoost, LightGBM.

🌐 Ray Serve

Online inference with dynamic batching, multi-model routing, and autoscaling on any cloud or on-prem.

MLflow — Experiment Tracking, Model Registry, and Versioning

The four-component ML lifecycle platform used across every major cloud

```
import mlflow

# 1. Log every training run
mlflow.set_experiment('loan-risk-v3')
with mlflow.start_run():
    mlflow.log_params({'n_estimators': 100, 'max_depth':
6})
    mlflow.log_metric('accuracy', 0.87)
    mlflow.sklearn.log_model(model, 'model')

# 2. Register to Model Registry
client = mlflow.MlflowClient()
client.create_registered_model('loan-risk')
client.create_model_version(
    name='loan-risk',
    source=run.info.artifact_uri
)

# 3. Promote to Production
client.transition_model_version_stage(
    name='loan-risk', version='3',
    stage='Production'
)
# 4. Query best run
best = mlflow.search_runs(order_by=['metrics.accuracy
DESC'])[0]
```

📊 Tracking

Log params, metrics, and artifacts for every run. Compare runs in the UI side by side.

📦 Models

Log any framework (sklearn, PyTorch, TF). Flavours make models framework-agnostic at serving time.

🗂️ Model Registry

Register, version, and transition models through Staging → Production lifecycle stages.

📋 Projects

Package ML code so anyone can reproduce it with mlflow run. Defines dependencies and entry points.

Comparing Ray, Kubeflow, and MLflow — When to Use What

Three tools for three different layers of the MLOps stack

BITS Pilani DIGITAL
Excellence made yours

⚡ Ray
Compute Layer

✅ **Use when**

Teams needing parallelism inside a stage — HPO, distributed training, fast inference.

⚠️ **Avoid when**

Pipeline scheduling. Ray parallelises work inside a step — it doesn't orchestrate steps.

📊 MLflow
Tracking Layer

✅ **Use when**

Any ML team logging runs, comparing experiments, and registering models. No infrastructure required.

⚠️ **Avoid when**

Pipeline orchestration or compute scaling — MLflow tracks, it doesn't run or schedule.

Kubeflow
Orchestration Layer

✅ **Use when**

Large ML teams on Kubernetes needing full lifecycle: train, tune, serve, and track in one platform.

⚠️ **Avoid when**

Small teams without K8s expertise. Infrastructure overhead is significant.

TOPIC 4

04

Automated Retraining & Drift Detection

When the world changes, your model must change with it

Concept Drift vs Data Drift — When to Trigger Retraining

Not all drift is the same — misidentifying it leads to the wrong response

BITS Pilani DIGITAL
Excellence made yours

📊 Data Drift (Covariate Shift)

WHAT: Input feature distribution changes — X→Y relationship stays the same.

EXAMPLE: Income scores shift upward after an economic boom.

DETECT: KS test or PSI per feature against training baseline.

RESPOND: Retrain on new data — model structure is fine, it just needs fresh training distribution.

In this week's lab: KS p-value < 0.05 on any feature triggers automated retraining.

🔄 Concept Drift (Label Shift)

WHAT: The X→Y relationship changes — even identical inputs now have different correct outputs.

EXAMPLE: Borrowers who once defaulted no longer do due to regulatory changes.

DETECT: Monitor accuracy against ground truth labels over time. Requires a feedback loop.

RESPOND: May need model architecture changes — not just retraining on new data.

⚠️ Ground truth labels often arrive weeks later — concept drift detection is always delayed.

Setting Up Automated Retraining Triggers

Four strategies and when each makes sense

⏱️ Schedule-Based

Retrain on a fixed cron schedule regardless of drift.

Use when: stable, low-velocity data. ⚠️ May trigger unnecessarily.

📊 Drift-Based

KS p < 0.05 on features fires the retraining pipeline.

Use when: real-time inference, fast-changing environments. ✅ No labels needed.

📈 Performance-Based

Accuracy drops below gate threshold → retrain.

Use when: ground truth labels available quickly. ⚠️ Signal is delayed by label lag.

📦 Volume-Based

N new labelled rows arrived → retrain.

Use when: active learning or batch labelling pipelines. ⚠️ Misses distribution shift.

Triggering Pipeline Retraining on Data Drift

The automated retraining loop implemented in this week's notebook

Capture Inference Requests	KS Test vs Baseline	CloudWatch Alarm Fires	Retrain New Model	Gate & Promote
Capture 100–500 inference requests to S3 each monitoring window.	scipy.stats.ks_2samp() per feature — flag if p < 0.05.	DriftedFeatureCount >= 2 fires the alarm. In prod: SNS → Lambda → trigger DAG.	Train on fresh data. Log run to SageMaker Experiments. Save artifact to dev/.	New accuracy > gate: promote to staging/prod. Else: rollback to v_prev.

Pipeline-Level Model Versioning and Automated Rollback

Production models must be replaceable and recoverable at any time

📦 Model Versioning Pattern

Every trained model is registered with an immutable version tag.

Each version stores:
- Artifact S3 path + SHA256 hash
- Training run_id and dataset reference
- Accuracy, F1, and drift trigger event
- Promotion timestamp and environment

Keep N=3 previous approved versions at all times.
This enables instant rollback to any prior version.

↩️ Automated Rollback Logic

Rollback triggers:
- New model accuracy < old model − margin
- Gate passed but shadow test failed
- Production error rate spikes after promotion
- Data validation fails on new training batch

Rollback pattern:
1. Query registry for last Approved version
2. Re-point prod prefix to v_prev artifact
3. Update registry status: v_new = Rejected
4. Alert on-call with reason and ARN

TOPIC 5

05

Hands-on Setup

Setting up MLflow and Kubeflow locally

Setting Up MLflow and Kubeflow Environment

MLflow runs in minutes locally — Kubeflow runs on Kubernetes

1 Install and Start MLflow

```
pip install mlflow
# Start local tracking server
mlflow server --backend-store-uri sqlite:///mlflow.db \
  --default-artifact-root ./mlruns --port 5000
# UI at: http://localhost:5000
```

2 Track a Run in Python

```
import mlflow
mlflow.set_experiment('loan-risk')
with mlflow.start_run():
    mlflow.log_param('lr', 0.01)
    mlflow.log_metric('accuracy', 0.88)
    mlflow.sklearn.log_model(model, 'model')
```

3 Install Kubeflow (Minikube path)

```
# 1. minikube start --cpus 4 --memory 8192
# 2. kubectl apply -k github.com/kubeflow/
#    pipelines/manifests/kustomize/...
# 3. kubectl port-forward -n kubeflow \
#    svc/ml-pipeline-ui 8080:80
```

4 Submit a Kubeflow Pipeline

```
import kfp
client = kfp.Client(host='http://localhost:8080')
client.create_run_from_pipeline_func(
    loan_pipeline,
    arguments={'data_path': 's3://my-bucket/data'}
)
```

TOPIC 6

06

Lesson Summary

Week 3 recap and key takeaways

Week 3 Recap — Advanced Pipeline Orchestration

BITS Pilani DIGITAL
Excellence made yours

Five production principles to take forward

1

Environments Are Not Optional

Dev → staging → prod isolation stops bad models reaching users. Promotion = s3.copy_object() + gate check.

2

Kubeflow Is ML-Native Orchestration

Each step runs in its own container. Artifact lineage and metadata tracking are built-in, not bolted on.

3

MLflow Is the Experiment Registry

Log every run — params, metrics, artifacts. Model Registry tracks versions, approval status, and rollback history.

4

Drift Detection Drives Retraining

KS test per feature flags covariate shift. When $p < 0.05$ on ≥ 2 features, the retraining pipeline fires automatically.

5

Rollback Is a Feature, Not a Fallback

v2 replaces v1 only if it improves accuracy by the defined margin. Otherwise v1 stays — automatically.

WHAT'S NEXT · WEEK 4

BITS Pilani DIGITAL
Excellence made yours

Containerization Fundamentals

Docker · Multi-stage Builds · GPU Containers · ECR · Container Security for ML Workloads

Docker Fundamentals

Images, containers, Dockerfile syntax, layers

Container Lifecycle

Build, tag, push, run — hands-on workflow

GPU Containers

CUDA base images and NVIDIA toolkit

Container Security

Non-root, ECR scanning, immutable tags